simple_net_scanner

```
Simple Network Scanner — beginner-friendly Python tool
Project: simple_net_scanner
File: simple_net_scanner.py
License: MIT
```

Project summary

`simple_net_scanner` is a small, beginner-friendly Python network scanner intended for learning and home-lab use. It performs a ping sweep (to detect live hosts) and a fast TCP connect scan against a set of common ports for each live host. The tool is intentionally simple, easy to read, and ready to publish on GitHub.

Important: Only scan networks and hosts you own or have explicit permission to test. Unauthorized scanning may be illegal and unethical.

Included files (ready for GitHub)

- `README.md` (this document)
- `simple_net_scanner.py` (main scanner script)
- `LICENSE` (MIT)
- `.gitignore`
- `logo.txt` (ASCII logo shown below)

ASCII logo (`logo.txt`)

```
 ____  _                 _ _   _   _      _   _
/ ___|| | ___  _   _  __| (_) | \ | | ___| |_(_) ___  _ __  ___
\___ \| |/ _ \| | | |/ _` | | |  \| |/ _ \ __| |/ _ \| '_ \/ __|
 ___) | | (_) | |_| | (_| | | | |\  |  __/ |_| | (_) | | | \__ \
|____/|_|\___/ \__,_|\__,_|_|_|_| \_|\___|\__|_|\___/|_| |_|___/

simple_net_scanner — beginner project
```

`simple_net_scanner.py` (complete, ready-to-run)

```
#!/usr/bin/env python3
"""
simple_net_scanner.py
A beginner-friendly network scanner:
 - Ping-sweep hosts in a CIDR (or single IP)
 - For live hosts, run a TCP connect scan on common ports

Usage:
    python simple_net_scanner.py 192.168.1.0/24
    python simple_net_scanner.py 192.168.1.10

This script uses subprocess `ping` for host discovery (works cross-platform) and
socket.connect for port checks.
Run as a normal user for small networks. For large or high-speed scans consider
running on a lab machine.
"""

import sys
import ipaddress
import platform
import subprocess
import socket
import concurrent.futures
import argparse

# Common ports to check (small set for speed)
COMMON_PORTS = [21, 22, 23, 25, 53, 80, 110, 139, 143, 443, 445, 587, 3306,
3389, 8080]

TIMEOUT = 1.0  # socket timeout in seconds
PING_COUNT = 1


def is_host_alive(ip: str) -> bool:
    """Ping the host once. Returns True if host replies.
       Uses system ping via subprocess for cross-platform simplicity."""
    param = '-n' if platform.system().lower() == 'windows' else '-c'
    cmd = ['ping', param, str(PING_COUNT), '-W', '1', ip] if
platform.system().lower() != 'windows' else ['ping', param, str(PING_COUNT), ip]
    try:
        proc = subprocess.run(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        return proc.returncode == 0
    except Exception:
```

```
        return False


def scan_port(ip: str, port: int) -> (int, bool):
    """Attempt TCP connect to (ip, port). Returns (port, is_open)."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(TIMEOUT)
        try:
            s.connect((ip, port))
            return port, True
        except Exception:
            return port, False


def scan_host_ports(ip: str, ports=COMMON_PORTS) -> dict:
    """Scan common ports on a host and return a dict of open ports."""
    open_ports = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=50) as ex:
        futures = [ex.submit(scan_port, ip, p) for p in ports]
        for fut in concurrent.futures.as_completed(futures):
            port, is_open = fut.result()
            if is_open:
                open_ports.append(port)
    return {'ip': ip, 'open_ports': sorted(open_ports)}


def parse_targets(target_arg: str):
    """Accept a single IP or CIDR. Yield IP strings."""
    try:
        if '/' in target_arg:
            net = ipaddress.ip_network(target_arg, strict=False)
            for ip in net.hosts():
                yield str(ip)
        else:
            # single IP
            yield str(ipaddress.ip_address(target_arg))
    except ValueError as e:
        print(f"Error: invalid target '{target_arg}': {e}")
        sys.exit(1)


def main():
    parser = argparse.ArgumentParser(description='simple_net_scanner — ping
sweep + simple TCP port scan')
    parser.add_argument('target', help='IP or CIDR to scan, e.g. 192.168.1.0/24
or 192.168.1.10')
    parser.add_argument('-p', '--ports', help='Comma separated list of ports to
scan (default: common ports)', default=None)
```

```
    parser.add_argument('-t', '--timeout', help='socket timeout seconds
(default: 1)', default=TIMEOUT, type=float)
    args = parser.parse_args()

    global TIMEOUT
    TIMEOUT = float(args.timeout)

    ports = COMMON_PORTS
    if args.ports:
        try:
            ports = [int(x.strip()) for x in args.ports.split(',') if x.strip()]
        except ValueError:
            print('Port list invalid. Use comma separated integers like
22,80,443')
            sys.exit(1)

    targets = list(parse_targets(args.target))
    print(f"Scanning {len(targets)} addresses from target {args.target}")

    live_hosts = []
    # Parallelize pings
    with concurrent.futures.ThreadPoolExecutor(max_workers=100) as ex:
        future_to_ip = {ex.submit(is_host_alive, ip): ip for ip in targets}
        for fut in concurrent.futures.as_completed(future_to_ip):
            ip = future_to_ip[fut]
            try:
                alive = fut.result()
            except Exception:
                alive = False
            if alive:
                print(f"Host alive: {ip}")
                live_hosts.append(ip)

    if not live_hosts:
        print('No live hosts found.')
        return

    # Scan ports for live hosts
    results = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=50) as ex:
        futures = [ex.submit(scan_host_ports, ip, ports) for ip in live_hosts]
        for fut in concurrent.futures.as_completed(futures):
            results.append(fut.result())

    print('\nScan results:')
    for r in sorted(results, key=lambda x: x['ip']):
        ip = r['ip']
        if r['open_ports']:
```

```
            print(f"{ip}: Open ports -> {', '.join(str(p) for p in
r['open_ports'])}")
        else:
            print(f"{ip}: No common open ports found")


if __name__ == '__main__':
    main()
```

`.gitignore`

```
__pycache__/
*.pyc
.env
.DS_Store
```

`LICENSE` (MIT)

```
MIT License

Copyright (c) 2025 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

[...standard MIT text continued...]
```

(Replace `Your Name` and the year as appropriate before publishing.)

README: Usage examples and GitHub-ready text

Quick install & run

```
# clone (after creating repo on GitHub)
git clone https://github.com/<your-username>/simple_net_scanner.git
```

```
cd simple_net_scanner
# run (python3 recommended)
python3 simple_net_scanner.py 192.168.1.0/24
# or scan a single host and custom ports
python3 simple_net_scanner.py 192.168.1.10 -p 22,80,443
```

Example output

```
Scanning 254 addresses from target 192.168.1.0/24
Host alive: 192.168.1.1
Host alive: 192.168.1.100
Host alive: 192.168.1.101

Scan results:
192.168.1.1: Open ports -> 22, 80
192.168.1.100: No common open ports found
192.168.1.101: Open ports -> 445
```

Repo description (short blurb for GitHub)

> A small, beginner-friendly Python network scanner that performs ping sweeps and simple TCP connect scans on common ports. Suitable for learning, home labs, and demos. **Do not use on networks without permission.**

How to make the repo GitHub-ready (quick commands)

```
# from inside project folder
git init
git add .
git commit -m "Initial commit: simple_net_scanner"
# create a repo on GitHub web UI named simple_net_scanner and then
git remote add origin git@github.com:<your-username>/simple_net_scanner.git
git branch -M main
git push -u origin main
```

Contribution & Issues

Feel free to open issues or PRs for: - Additional port lists - ICMP-less host discovery (ARP scanning for local network) - Output formats (JSON, CSV) - Rate-limiting and delay options for stealth - Windows-specific improvements

---

Security & Legal

Only run this tool against systems/networks you own or are authorized to test. The author is not responsible for misuse.

Next steps / optional extras you may want me to add

- Add a `requirements.txt` and packaging for PyPI
- Add JSON/CSV report output and sample reports
- Add unit tests and CI configuration (GitHub Actions)
- Create a small webpage (README badges, demo GIF)

I prepared this repository content so you can copy files into a local folder and push to GitHub. Replace placeholder fields (your name, GitHub username) before publishing.